

LẬP TRÌNH JAVA



Giảng viên: ThS Huỳnh Vĩnh Phát

Biên soạn: TS Trần Duy Thanh

Bài Học

Vì sao phải lưu tập tin



Vì sao phải lưu tập tin

- ❖ Các dữ liệu ta đang thao tác là trên thanh RAM → mất dữ liệu khi tắt phần mềm, tắt máy
- ❖ Nên cần phải lưu trữ vào thiết bị lưu trữ vĩnh cửu rồi nạp trở lại bộ nhớ.

Vì sao phải lưu tập tin

Bộ nhớ trong



Dữ liệu trong phần mềm mà ta đang thao tác

Bộ nhớ ngoài



Các loại tập tin lưu trữ

- ❖ Có rất nhiều loại, trong khóa học này ta quan tâm tới 2 loại: TextFile và Serialize File

Bài Học

Xử lý tập tin với TextFile



Các lớp thư viện lưu/đọc TextFile

```
import java.io.BufferedReader;  
import java.io.BufferedWriter;  
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.InputStreamReader;  
import java.io.OutputStreamWriter;
```



Kỹ thuật lưu trữ tập tin với TextFile

```
FileOutputStream fos=new FileOutputStream("d:/data.txt");  
OutputStreamWriter osw=new OutputStreamWriter(fos, "UTF-8");  
BufferedWriter bw=new BufferedWriter(osw);  
  
bw.write("Obama");  
bw.newLine();  
bw.write("Putin");  
bw.newLine();  
bw.write("Kim Jong Un");  
  
bw.close();  
osw.close();  
fos.close();
```


Kỹ thuật đọc tập tin với TextFile

```
FileInputStream fis=new FileInputStream("d:/data.txt");  
InputStreamReader isr=new InputStreamReader(fis, "UTF-8");  
BufferedReader br=new BufferedReader(isr);
```

```
String line1=br.readLine();  
String line2=br.readLine();  
String line3=br.readLine();
```

```
br.close();  
isr.close();  
fis.close();
```

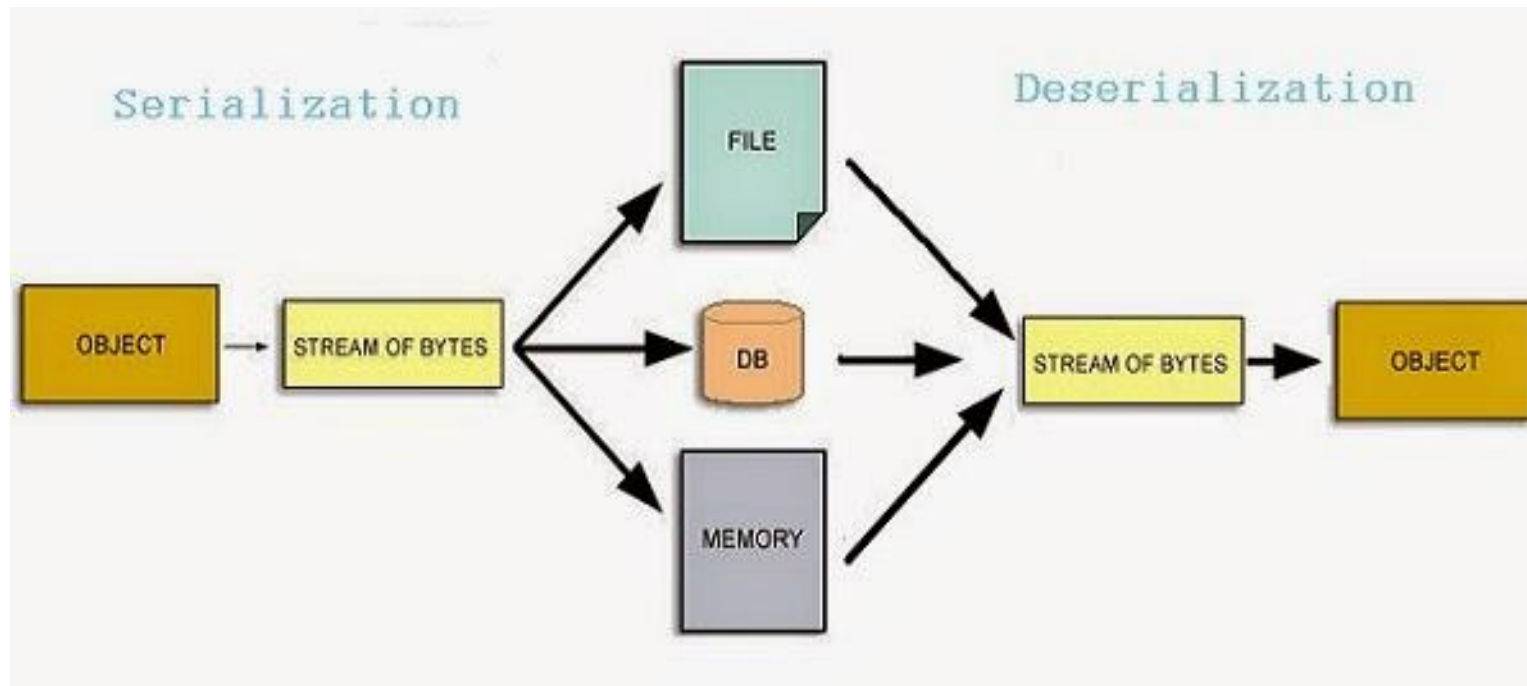
Bài Học

Xử lý tập tin với Serialize File



Nội dung bài học

Serialization đơn giản chỉ là chuyển từ một object tồn tại thành một mảng byte. Mảng byte này đại diện cho class của object, phiên bản của object, và trạng thái của object. Mảng byte này có thể được sử dụng giữa các máy ảo JVM đang chạy cùng code truyền / đọc các object. Ngược lại Deserialization là quá trình xây dựng lại các byte vào một đối tượng.



```
public class KhachHang implements Serializable{  
    private String ma;  
    private String ten;  
    public String getMa() {  
        return ma;  
    }  
    public void setMa(String ma) {  
        this.ma = ma;  
    }  
    public String getTen() {  
        return ten;  
    }  
    public void setTen(String ten) {  
        this.ten = ten;  
    }  
    public KhachHang(String ma, String ten) {  
        super();  
        this.ma = ma;  
        this.ten = ten;  
    }  
}
```



Các lớp thư viện lưu/đọc Serialize File

```
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.ObjectInputStream;  
import java.io.ObjectOutputStream;
```

Kỹ thuật lưu trữ tập tin với Serialize File

```
FileOutputStream fos=new FileOutputStream("d:/data.dat");  
ObjectOutputStream oos=new ObjectOutputStream(fos);  
KhachHang kh=new KhachHang (1,"Nguyễn Văn Tèo");  
oos.writeObject(kh);  
oos.close();  
fos.close();
```

Kỹ thuật đọc tập tin với Serialize File

```
FileInputStream fis=new FileInputStream("d:/data.dat");  
ObjectInputStream ois=new ObjectInputStream(fis);  
Object data=ois.readObject();  
KhachHang kh=(KhachHang) data;  
ois.close();  
fis.close();
```

Bài Học

Bài tập rèn luyện với TextFile



Nội dung bài học

- ❖ Viết phần mềm quản lý Khách Hàng dùng TextFile:
- ✓ Khách hàng : Mã, tên, năm sinh
- ✓ Cho nhập danh sách Khách Hàng từ bàn phím
- ✓ Cho xuất danh sách khách hàng ra màn hình
- ✓ Cho lưu danh sách khách hàng vào ổ cứng
- ✓ Cho đọc danh sách khách hàng từ ổ cứng

Bài Học

Bài tập rèn luyện với Serialize File



Nội dung bài học

- ❖ Viết phần mềm quản lý Danh Bạ dùng SerializeFile:
- ✓ Danh bạ cần có các thông tin : tên, số phone
- ✓ Cho nhập danh sách Danh Bạ từ bàn phím
- ✓ Cho xuất danh sách Danh Bạ ra màn hình
- ✓ Cho lưu danh sách Danh Bạ vào ổ cứng
- ✓ Cho đọc danh sách Danh Bạ từ ổ cứng

Bài Học

Khái niệm về đa tiến trình và cơ chế hoạt động



Nội dung bài học

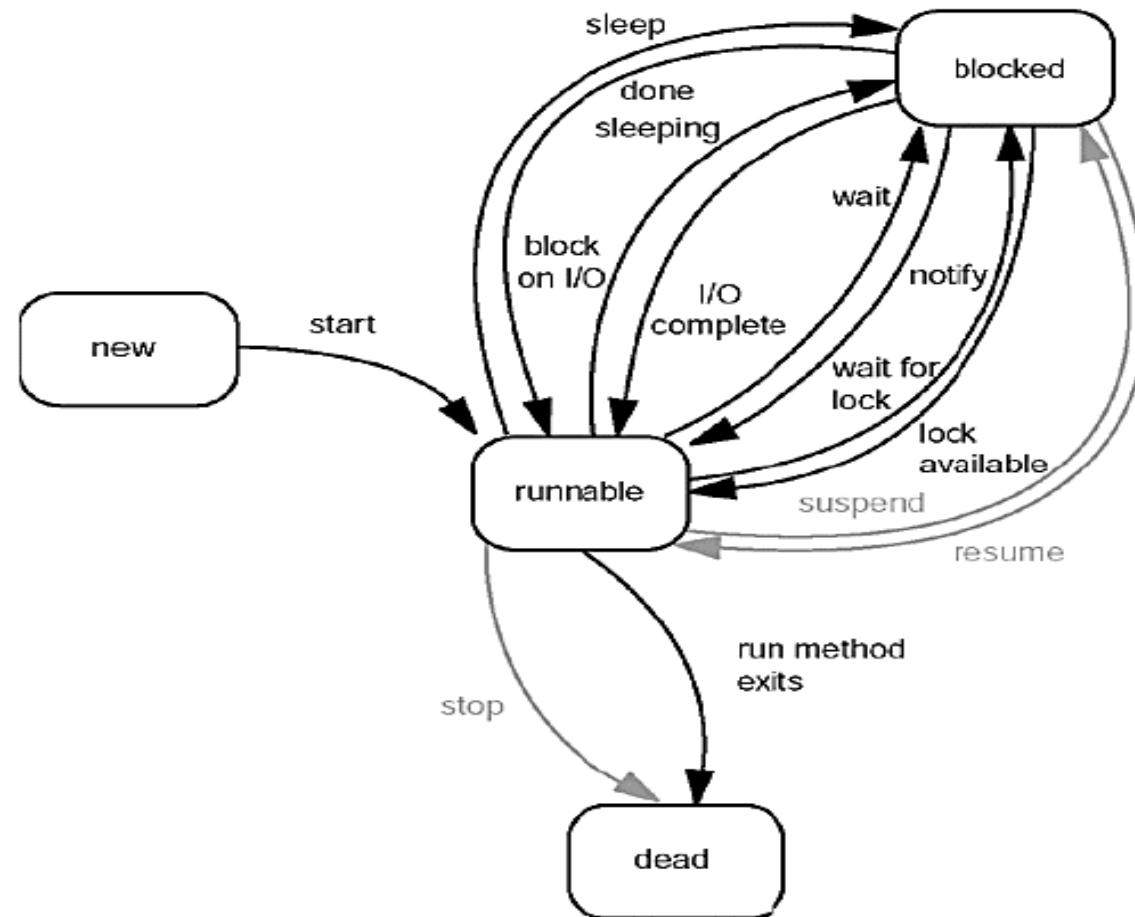
- ❖ Khái niệm về đa tiến trình
- ❖ Trạng thái của Thread
- ❖ Độ ưu tiên của Thread
- ❖ Các kỹ thuật tạo đa tiến trình trong Java

Khái niệm về đa tiến trình

- ❖ Một chương trình đa luồng chứa hai hoặc nhiều phần mà có thể chạy đồng thời và mỗi phần có thể xử lý tác vụ khác nhau tại cùng một thời điểm, để sử dụng tốt nhất các nguồn có sẵn, đặc biệt khi máy tính của bạn có nhiều CPU.
- ❖ Theo định nghĩa, đa nhiệm (multitasking) là khi nhiều tiến trình chia sẻ nguồn xử lý chung ví dụ như một CPU. Thread có thể chia nhỏ các hoạt động riêng biệt bên trong một ứng dụng đơn thành các luồng (thread) riêng lẻ. Mỗi một thread có thể chạy song song. OS phân chia thời gian xử lý không chỉ trong các ứng dụng khác nhau, mà còn trong mỗi luồng bên trong một ứng dụng.
- ❖ Thread cho Ta khả năng viết một chương trình mà có nhiều hoạt động có thể phát sinh đồng thời.

Trạng thái của Thread

- ❖ Một thread đi qua các giai đoạn khác nhau trong vòng đời của nó.



Trạng thái của Thread

- **New:** Một thread mới bắt đầu vòng đời của nó trong trạng thái new. Nó tồn tại trong trạng thái này tới khi chương trình bắt đầu thread này. Nó cũng được xem như là một thread mới sinh.
- **Runnable:** Sau khi một thread mới sinh ra được bắt đầu, thread trở thành runnable. Một thread trong trạng thái này được xem như đang thực hiện tác vụ của nó.
- **Blocked:** Tiến trình bị block tạm thời, để cho các tiến trình khác có cơ hội xen vào.
- **Dead:** Một thread trong trạng thái runnable có thể đi vào trạng thái Dead khi nó hoàn thành tác vụ của nó hoặc nó chấm dứt.

Độ ưu tiên của Thread

- Mỗi thread trong Java có một quyền ưu tiên mà giúp hệ điều hành xác định thứ tự thread nào được ghi lịch trình.
- Quyền ưu tiên của thread trong Java là một dãy giữa `MIN_PRIORITY` (hằng số 1) và `MAX_PRIORITY` (hằng số 10). Theo mặc định, mỗi thread được cung cấp một quyền ưu tiên `NORM_PRIORITY` (hằng số 5).
- Các thread với quyền ưu tiên cao hơn là quan trọng hơn với một chương trình và nên được cấp phát thời gian bộ vi xử lý trước các thread có quyền ưu tiên thấp hơn. Tuy nhiên, các quyền ưu tiên của thread bảo đảm thứ tự trong đó các thread thực thi và phụ thuộc rất nhiều vào platform.

Các kỹ thuật tạo đa tiến trình trong Java

- Tạo Thread từ kế thừa lớp **Thread** trong Java
- Tạo thread từ implements Interface **Runnable** trong Java

Cách tạo Lớp kế thừa từ Thread

- Tạo Thread từ kế thừa lớp **Thread** trong Java

```
public class TheFirstThread extends Thread {  
    public TheFirstThread()  
    {  
        System.out.println("The First Thread Is Created!");  
    }  
    public void run() {  
        for(int i=0;i<10;i++)  
        {  
            System.out.println("Th #1:"+i);  
        }  
    }  
}
```

1. Override method **run** of Thread class.
2. The code inside is the task assign to thread when running.

Cách sử dụng đa tiến trình với Thread

```
TheFirstThread th1=new TheFirstThread();  
th1.start();
```

Cách tạo Lớp implements từ Runnable

- Tạo thread từ implements Interface **Runnable** trong Java

```
public class TheSecondThread implements Runnable {  
    public TheSecondThread()  
    {  
        System.out.println("The First Second Is Created!");  
    }  
    public void run() {  
        for(int i=0;i<10;i++)  
        {  
            System.out.println("Th #2:"+i);  
        }  
    }  
}
```

1. Override method **run** of Thread class.
2. The code inside is the task assign to thread when running.

Cách sử dụng đa tiến trình với Runnable

```
TheSecondThread run2=new TheSecondThread();  
Thread th2=new Thread(run2);  
th2.start();
```

Stack Overflow

AboutProductsFor Teams

Search...

Home

Questions

AI Assist

Staging Ground

Tags

Saves

Challenges

Chat

Articles

Users

Companies

COLLECTIVES +
Communities for your favorite technologies.
[Explore all Collectives](#)

STACK INTERNAL +
Bring the best of human thought and AI automation together at your work.
[Learn more](#)

"implements Runnable" vs "extends Thread" in Java

Asked 17 years, 2 months ago Modified 11 months ago Viewed 791k times

From what time I've spent with threads in Java, I've found these two ways to write threads:

2380 With **implements Runnable** :

```
public class MyRunnable implements Runnable {
    public void run() {
        //Code
    }
}
//Started with a "new Thread(new MyRunnable()).start()" call
```

Or, with **extends Thread** :

```
public class MyThread extends Thread {
    public MyThread() {
        super("MyThread");
    }
    public void run() {
        //Code
    }
}
//Started with a "new MyThread().start()" call
```

Is there any significant difference in these two blocks of code?

42 Answers

Sorted by: Highest score (default)

1 2 Next

- 1850
- Yes: implements **Runnable** is the preferred way to do it, IMO. You're not really specialising the thread's behaviour. You're just giving it something to run. That means [composition](#) is the *philosophically* "purer" way to go.

In *practical* terms, it means you can implement **Runnable** and extend from another class as well... and you can also implement **Runnable** via a lambda expression as of Java 8.

Share Edit Follow Flag

edited Oct 26, 2020 at 14:02

answered Feb 12, 2009 at 14:32

Jon Skeet
1.5m ● 895 ● 9.3k ● 9.4k

31 Comments

- Add a comment
- hooknc Over a year ago
Exactly, well put. What behavior are we trying to overwrite in Thread by extending it? I would argue most people are not trying to overwrite any behavior, but trying to use behavior of Thread.
- Nacho Coloma Over a year ago

Bài Học

Bài tập tự rèn luyện đa tiến trình



Nội dung bài học

❖ Tạo một Thread kế thừa từ Thread class

```
public class MyThread extends Thread {  
    public static void main(String[] args) {  
        MyThread th=new MyThread();  
        th.start();  
        System.out.println("This is the main thread");  
    }  
    public void run() {  
        while(true)  
        {  
            try  
            {  
                System.out.println("This is the child Thread");  
                sleep(1000);  
            }  
            catch (InterruptedException ex)  
            {  
                ex.printStackTrace();  
            }  
        }  
    }  
}
```

Nội dung bài học

❖ Tạo một Thread implements từ Runnable interface

```
public class MyThread2 implements Runnable {
    public static void main(String[] args) {
        MyThread2 myRunnable=new MyThread2();
        Thread th2=new Thread(myRunnable);
        th2.start();
        System.out.println("This is the main thread");
    }
    public void run() {
        while(true)
        {
            try
            {
                System.out.println("This is the child Thread");
                Thread.sleep(1000);
            }
            catch (InterruptedException ex)
            {
                ex.printStackTrace();
            }
        }
    }
}
```

Nội dung bài học

❖ Hiển thị các trạng thái trong Thread

```
public class ThreadStateDemo extends Thread {  
    static Thread th;  
    public static void main(String[] args) {  
        th=new ThreadStateDemo();  
        System.out.println("Thread th is born");  
        System.out.println("Thread th is ready");  
        th.start();  
    }  
    public void run() {  
        System.out.println("Thread th is running");  
    }  
}
```

Nội dung bài học

❖ Kiểm tra `currentThread`, `acctiveCount` và `isAlive`

```
public class ThreadTest implements Runnable{
    Thread objTh;
    public ThreadTest()
    {
        System.out.println(Thread.currentThread().getName());
        objTh=new Thread(this);
        System.out.println("Thread objTh isAlive =" +objTh.isAlive());
        objTh.setName("New Thread");
        objTh.start();
    }
    public void run() {
        System.out.println(Thread.currentThread().getName());
        System.out.println(Thread.activeCount());
        System.out.println("Thread objTh isAlive =" +objTh.isAlive());
    }
    public static void main(String[] args) {
        new ThreadTest();
    }
}
```

Nội dung bài học

- ❖ Thiết lập độ ưu tiên cho Thread (dùng phương thức `setPriority`), có 3 độ ưu tiên căn bản : `MAX_PRIORITY` (10), `NORM_PRIORITY`(5), `MIN_PRIORITY`(1). Chúng ta có thể thiết lập độ ưu tiên tùy ý từ 1 tới 10.

Nội dung bài học

```
package chap1.multithreading;
import java.util.Random;
public class ThreadPriority extends Thread{
    int num;
    public ThreadPriority(int num)
    {
        this.num=num;
    }
    public void run() {
        for(int i=0;i<num;i++)
        {
            System.out.println("Count = "+i);
        }
        System.out.println(currentThread().getName());
        System.out.println("Priority =" +getPriority());
    }
    public static void main(String[] args) {
        Random rd=new Random();
        for(int i=0;i<=2;i++)
        {
            ThreadPriority p=new ThreadPriority(rd.nextInt(5)+1);
            p.setPriority(rd.nextInt(10)+1);
            p.setName("ct"+i);
            p.start();
        }
    }
}
```



Nội dung bài học

```
package chap1.multithreading;
public class PriorityThread extends Thread{
    public PriorityThread(String name,int priority)
    {
        setName(name);
        setPriority(priority);
        start();
    }
    public void run() {
        System.out.println("Name="+getName() + " - Priority =" +getPriority());
    }
    public static void main(String[] args) throws InterruptedException {
        PriorityThread p1=new PriorityThread("Thread 113", Thread.MIN_PRIORITY);
        PriorityThread p2=new PriorityThread("Thread 114", Thread.NORM_PRIORITY);
        PriorityThread p3=new PriorityThread("Thread 115", Thread.MAX_PRIORITY);
        p1.join();
        p2.join();
        p3.join();
    }
}
```

END

